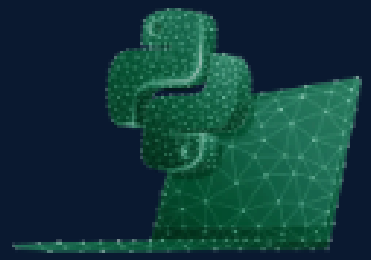


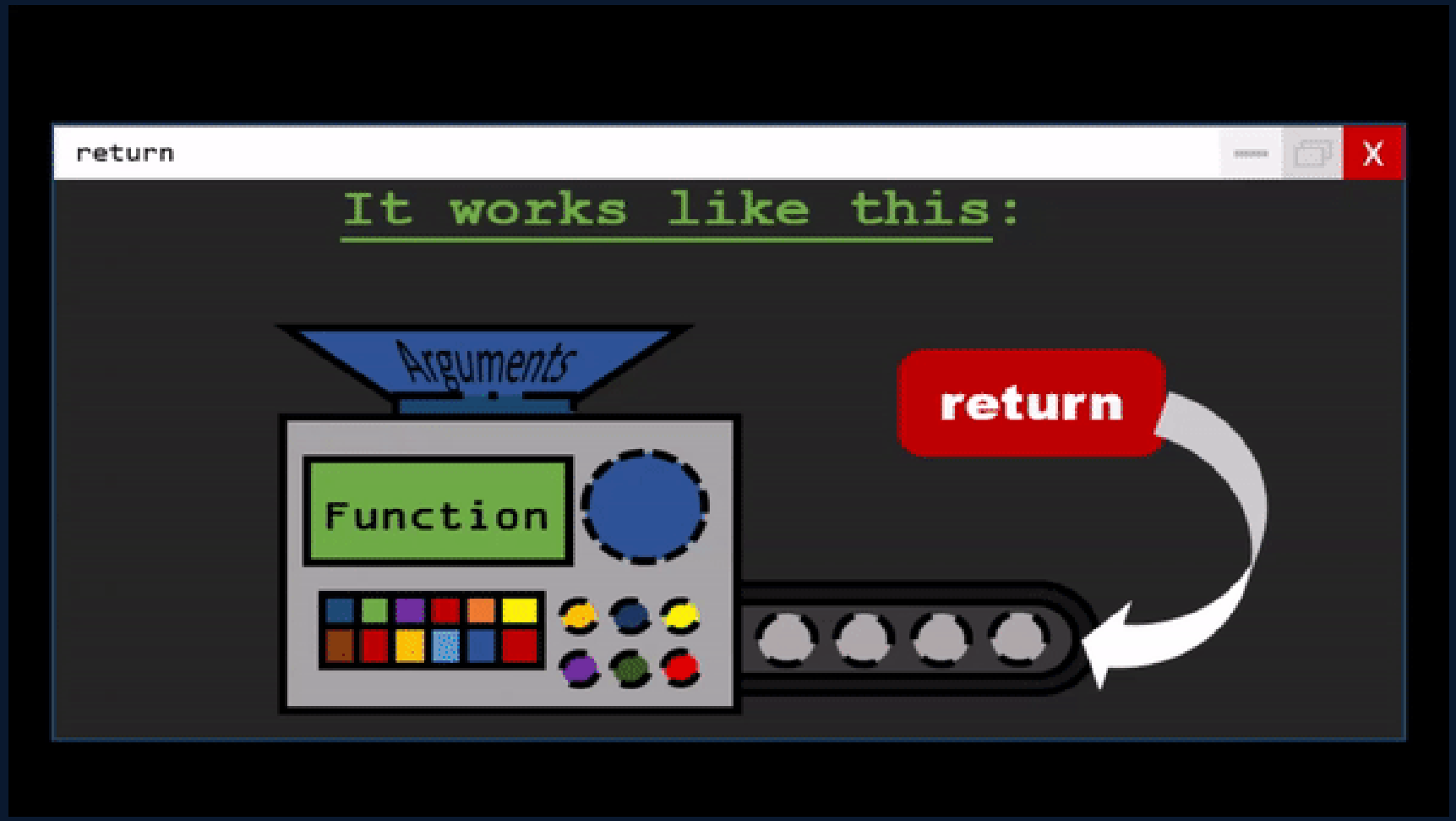
Python Functions

Making Code Simple, Reusable, and Organized

Presented by:
Amit Roy
MSc Student at IIT

The Basics

What are functions and why should we use them?

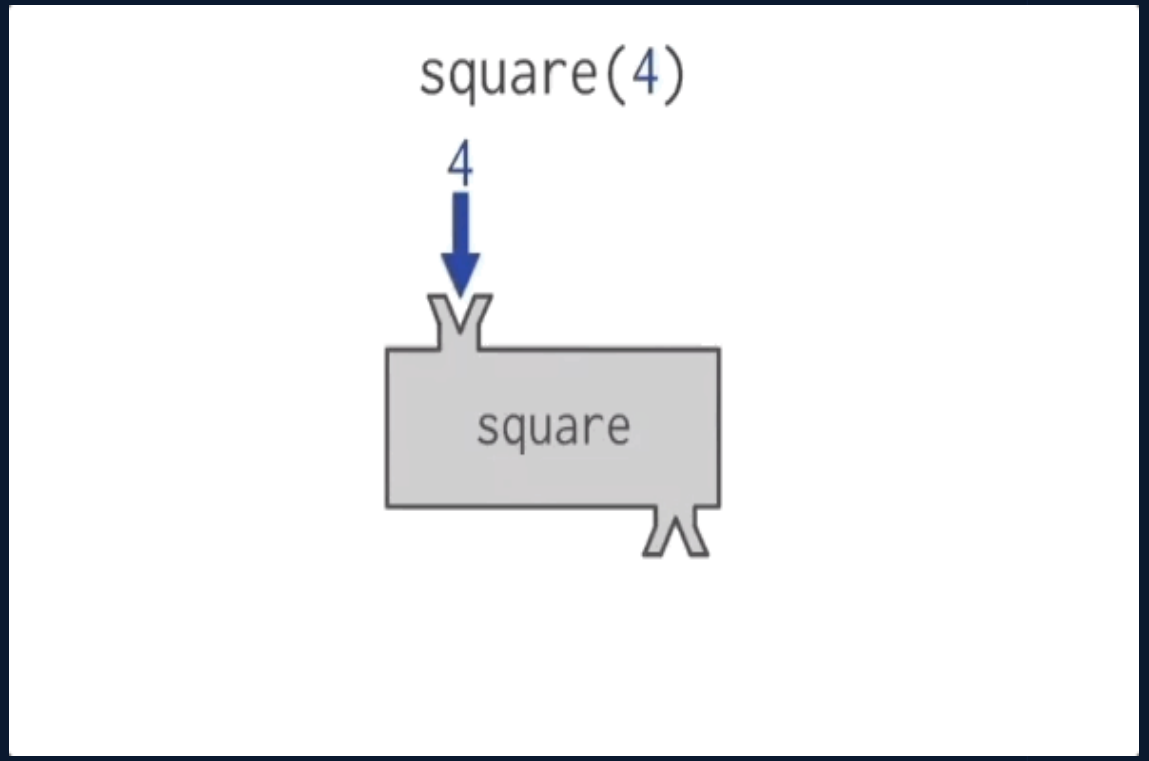


THE FUNCTION MACHINE

What is a Function?

Think of a function as a **Kitchen Blender**. You give it ingredients (Inputs), it performs a specific task (Mixing), and it gives you a smoothie (Output).
Instead of manually chopping fruit every time, you just press the "Blender" button.

- Reusable blocks of code
- Performs one specific task
- Makes code easier to read



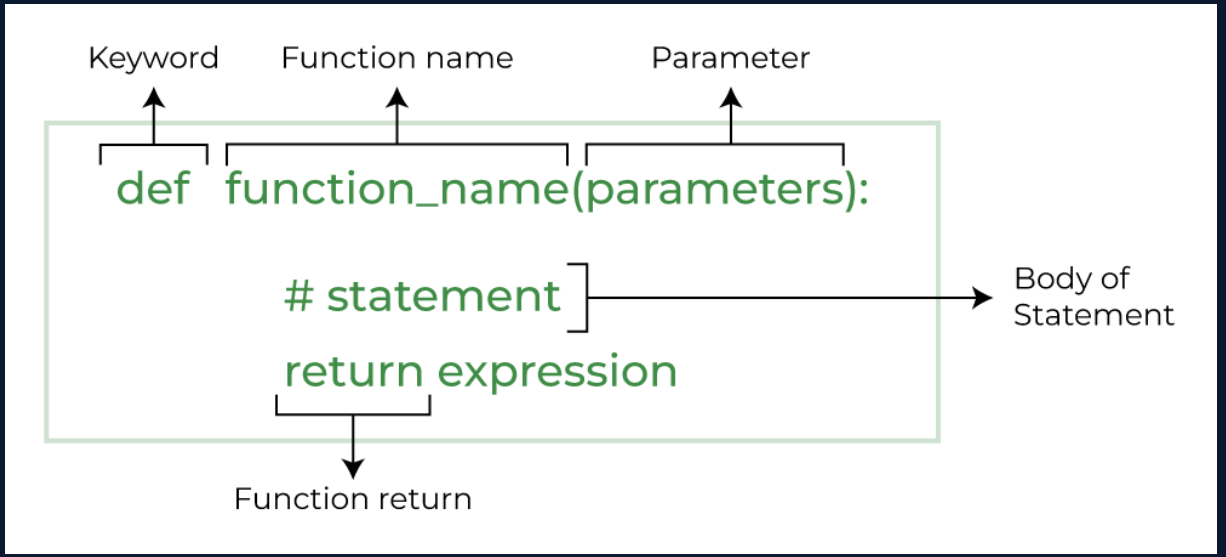
STRUCTURE OF A FUNCTION

- **def keyword**
  Used to define a function.

- **Function Name**
  The name (identifier) used to call the function.

- **Parameters (Optional)**
  Variables inside parentheses that receive input values.

- **Colon (:)**
  Indicates the beginning of the function body.



- **Function Body**
  An indented block of code that executes when the function is called.

- **return Statement (Optional)**
  Sends a value back to the caller.

Example (Like 8.2, 8.3)

```
def greet():
    print("Hello! Welcome to the class.")

greet()
```

Output

```
Hello! Welcome to the class.
```

Function Parameters (Like 8.4)

```
# Function with a single parameter
def greet(name):
    print("Hello", name)

# Calling the function and passing “Amit" as the argument
greet(“Amit")
```

Output

```
Hello Alice
```

Function Multiple Parameters (Like 8.4)

```python
# Function with multiple parameters
def add_numbers(a, b):
    print("The sum is:", a + b)

# Passing two numbers to the function
add_numbers(10, 5)
```

Output

```
The sum is: 15
```

Default Parameter (Like 8.6)

```python
# Setting "Guest" as the default value for the 'name' parameter
def greet(name="Guest"):
    print("Welcome,", name)

# Python uses the default "Guest")
greet()

# Calling with an argument (Overrides the default value)
greet("Amit")
```

Output

```
Welcome, Guest
Welcome, Amit
```

Return Statement (Like 8.5)

```
def scquare(x):
    return x * x
result = scquare(5)

print("The square of 5 is:", result)
```

Output

```
The square of 5 is: 25
```

Returning Multiple Values

```
def get_marks():
    x = 10
    y = 20
    return x, y

result = get_marks()
print (result)
print (result[0])
print (result[1])
```

Output

```
(10, 20)
10
20
```


OK প্যারা নাই!
GIFGARI.COM

Returning Values as a Dictionary

```python
def get_person_info():
    return {
        "name": "Alice",
        "age": 30,
        "city": "New York"
    }

info = get_person_info()

print (info["name"])
print(info["age"])
print(info["city"])
```

Output

```
Amit
90
Dhaka
```

Using a Dictionary as Function Parameter

```python
def print_user_info(user):
    print(f"Name: {user['name']}")
    print(f"Age: {user['age']}")
     print(f"City: {user.get('city', 'Unknown')}")

user_data = {"name": "Bob", "age": 25}
print_user_info (user_data)
```

Output

```
Name: Bob
Age: 25
City: Unknown
```

Keyword Arguments (Like 8.10)

```
def describe_pet(name, animal):
    print(f"I have a {animal} named {name}.")

describe_pet (animal="dog", name="Buddy")
```

Output

```
I have a dog named Buddy.
```

Variable Length Arguments (Like 8.11)

```
def print_numbers(*args):
    for num in args:
        print(num)

print_numbers(1, 2, 3, 4)
```

Output

```
1
2
3
4
```

```
def print_info( ** kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Amit", age=60)
```

Output

```
name: Amit
age: 60
```

Docstring

Use **Docstrings** (triple quotes """) to explain what your function does so others can use it easily.

```python
def greet (name):
    """ Print a greeting to the given name."""
    print (f"Hello, {name}!")
greet("Amit")

help(greet)
```

Output

```
Hello, Amit!
Help on function greet in module __main__:

greet(name)
    Print a greeting to the given name.
```

"Code is read much more often than it is written."

— Guido van Rossum

(Creator of Python)

Practice Problem

1. Write a function named `greet` that takes a person's name as a parameter and prints a greeting message, e.g., "Hello, Alice!".
2. Create a function `is_even` that accepts an integer and returns `True` if the number is even, and `False` otherwise.
3. Write a function `factorial` that calculates and returns the factorial of a given non-negative integer.

Practice Problem

4. Write a function `palindrome_check` that takes a string and returns `True` if the string is a palindrome (reads the same backward and forward), and `False` otherwise.
    - Example 1: `palindrome_check("madam")` returns `True`
    - Example 2: `palindrome_check("python")` returns `False`
    - Example 3: `palindrome_check("racecar")` returns `True`
5. Write a function `fizz_buzz` that takes an integer `n` and prints numbers from 1 to `n` with the following rules:
    - For multiples of 3, print "Fizz" instead of the number.
    - For multiples of 5, print "Buzz" instead of the number.
    - For multiples of both 3 and 5, print "FizzBuzz".
    - Prints the number itself otherwise

Expected Output: For `n = 15`, the output should be:

```
1 , 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz, 11, Fizz, 13, 14, FizzBuzz
```

Questions?

You are now ready to build your own Python functions!

</> Happy Coding